

DESAFIOS CONTEXTUALIZADOS — JAVA / POO

Lista de desafios para alunos — métodos, parâmetros, return, varargs, classes, objetos, construtores e this

Conteúdos permitidos

Variáveis; Scanner; operadores; if/else; switch; for; while; métodos; parâmetros; return; varargs; classes; objetos; atributos; construtores; this.

Orientação: a partir dos desafios de POO, evite colocar toda a lógica dentro do main. O main deve organizar o fluxo, enquanto classes e métodos devem executar as responsabilidades do sistema.

DESAFIO 01 — Cantina da escola

A cantina da escola deseja desenvolver um pequeno sistema para calcular o valor de uma compra. Cada produto possui nome, preço e quantidade.

- Crie a classe Produto com nome, preço e quantidade.
- Utilize um construtor para inicializar os dados.
- Crie calcularTotal(), retornando preço × quantidade.
- Desafio adicional: se o total for maior que R\$ 100, conceda 10% de desconto.

DESAFIO 02 — Boletim do aluno

A escola precisa de um programa para calcular o resultado dos alunos. Cada aluno possui nome, matrícula e três notas.

- Crie a classe Aluno.
- Utilize construtor.
- Crie calcularMedia(), verificarAprovacao() e mostrarDados().
- Regras: média ≥ 6 : Aprovado; média entre 4 e 5,9: Recuperação; média < 4 : Reprovado.
- O método verificarAprovacao() deve utilizar o resultado de calcularMedia().

DESAFIO 03 — Caixa da lanchonete

Uma lanchonete deseja controlar o valor dos pedidos. Cada pedido possui nome do cliente, produto, quantidade e preço unitário.

- Crie a classe Pedido.
- Utilize construtor.
- Crie calcularSubtotal(), calcularDesconto() e calcularTotal().
- Regras: subtotal $<$ R\$ 50: 0%; R\$ 50 até R\$ 99,99: 5%; R\$ 100 ou mais: 10%.

DESAFIO 04 — Conta bancária

Um banco precisa de uma aplicação simples para controlar uma conta. Cada conta possui titular, número e saldo.

- Crie ContaBancaria com construtor.
- Crie os métodos depositar(), sacar() e consultarSaldo().

- Não permitir depósito negativo, saque negativo ou saque maior que o saldo.
- Menu: 1-Depositar; 2-Sacar; 3-Consultar saldo; 4-Sair.
- O menu deve continuar até o usuário escolher 4.

DESAFIO 05 — Estacionamento

Um estacionamento deseja calcular o valor que cada veículo deverá pagar. Cada veículo possui placa, modelo e quantidade de horas estacionado.

- Crie Veiculo.
- Primeira hora: R\$ 10. Cada hora adicional: R\$ 5.
- Crie calcularValor().
- Desafio adicional: acima de 8 horas, tarifa fixa de R\$ 50.

DESAFIO 06 — Sistema de notas de uma turma

Um professor deseja calcular as médias de uma turma.

- Crie a classe Aluno com nome e matricula.
- Crie calcularMedia(double... notas) utilizando varargs.
- Crie pelo menos 5 objetos Aluno e calcule a média de cada um.

DESAFIO 07 — Loja de eletrônicos

Uma loja deseja controlar seus produtos. Cada produto possui código, nome, preço e estoque.

- Crie Produto.
- Métodos: adicionarEstoque(), vender(), consultarEstoque() e calcularValorEstoque().
- Não permitir quantidade negativa, venda maior que o estoque ou preço negativo.

DESAFIO 08 — Cadastro de funcionário

Uma empresa deseja cadastrar seus funcionários. Cada funcionário possui nome, cargo e salário.

- Crie Funcionario com construtor.
- Métodos: mostrarDados(), calcularAumento() e calcularSalarioAnual().
- calcularAumento() recebe um percentual e atualiza o salário.
- calcularSalarioAnual() retorna salário × 12.

DESAFIO 09 — Biblioteca escolar

A biblioteca precisa de um sistema para controlar empréstimos.

- Crie Livro com titulo, autor, codigo e disponivel.
- Métodos: emprestar(), devolver() e mostrarDados().
- Livro disponível pode ser emprestado; livro indisponível não pode ser emprestado.
- Só pode ser devolvido um livro que esteja emprestado.

DESAFIO 10 — Academia

Uma academia deseja controlar seus alunos. Cada aluno possui nome, idade, peso e altura.

- Crie Aluno.
- Crie calcularIMC(): $IMC = peso / (altura \times altura)$.
- Crie classificarIMC().
- Classificação: < 18,5 abaixo do peso; 18,5-24,9 peso normal; 25-29,9 sobrepeso; ≥ 30 obesidade.

DESAFIO 11 — Aplicativo de transporte

Um aplicativo precisa calcular o valor de uma corrida. A corrida possui passageiro e distância.

- Crie Corrida.
- Taxa inicial: R\$ 5; valor por km: R\$ 2,50.
- Fórmula: $valor = 5 + (distância \times 2,50)$.
- Se a distância for superior a 20 km, aplicar 10% de desconto.

DESAFIO 12 — Caixa eletrônico

Desenvolva um pequeno caixa eletrônico.

- Crie Conta com titular e saldo.
- Métodos: depositar(), sacar() e consultarSaldo().
- Menu: 1-Depositar; 2-Sacar; 3-Consultar saldo; 4-Sair.
- Utilize while ou do while.
- Impedir saque ≤ 0 , saque > saldo e depósito ≤ 0 .

DESAFIO 13 — Sistema de pedidos

Uma pequena empresa vende produtos pela internet.

- Crie as classes Produto e Pedido.
- Produto: nome e preço.
- Pedido: número, cliente, produto e quantidade.
- Métodos do pedido: calcularSubtotal(), calcularDesconto() e calcularTotal().
- Regras: subtotal < 100: 0%; 100-199,99: 5%; ≥ 200 : 10%.
- Desafio: trabalhar com um objeto dentro de outro objeto: Pedido $\rightarrow$ Produto.

DESAFIO 14 — Sistema de cursos

Uma escola técnica possui vários cursos.

- Crie Curso com nome e cargaHoraria.
- Crie Aluno com nome, matricula e curso.
- O construtor de Aluno deverá receber um objeto Curso.
- Crie mostrarDados() para apresentar os dados do aluno e do curso.

DESAFIO 15 — Sistema de estacionamento completo

Crie um sistema com as classes Veiculo e Estacionamento.

- Veiculo: placa, modelo e horaEntrada.
- Estacionamento: registrarEntrada(), registrarSaida() e calcularValor().
- Menu: 1-Registrar entrada; 2-Registrar saída; 3-Consultar veículo; 4-Sair.
- O aluno deverá definir como as classes irão se comunicar.

DESAFIO 16 — Sistema escolar completo

Desenvolva um sistema para uma escola técnica.

- Classes: Aluno, Curso, Disciplina e Professor.
- Aluno: nome, matricula e curso.
- Curso: nome e cargaHoraria.
- Disciplina: nome e cargaHoraria.
- Professor: nome e especialidade.
- Menu: cadastrar aluno, curso, disciplina e professor; mostrar aluno; mostrar curso; sair.
- Obrigatório: classes, objetos, atributos, construtores, this, métodos, parâmetros, return, Scanner, if/else, switch e repetição.

DESAFIO 17 — Problema aberto: você é o desenvolvedor

Uma empresa contratou sua equipe para desenvolver um pequeno sistema que resolva um problema real do cotidiano.

- Escolha um contexto: biblioteca, academia, escola, loja, restaurante, estacionamento, clínica, oficina, hotel ou cinema.
- Identifique as entidades, atributos e comportamentos.
- Defina os construtores necessários.
- Defina quais métodos retornam valores.
- Decida onde utilizar if, switch e repetição.
- O sistema deve possuir no mínimo 3 classes, com relacionamento entre pelo menos duas delas.
- Cada aluno deverá explicar uma parte do código durante a apresentação.